

Open Source Libraries for the *Workflow Graph* in Delphi 12.2

Evaluation of available Delphi-native and portable open source components for state machine engine, graph storage, JSON serialization, and supporting infrastructure

Context. The workflow graph described in Parts I and II requires four implementation concerns: (1) a **state machine engine** that executes nodes, fires transitions, and evaluates context-based guard conditions; (2) a **graph data structure and storage** mechanism for defining and persisting the workflow; (3) **JSON serialization** for the execution context and HTTP communication with the RPC gateway; and (4) **supporting infrastructure** — dependency injection and event bus — that holds the layered architecture together. Each concern has viable open source options in the Delphi ecosystem. Delphi 12.2 Community Edition supports all libraries listed here; CE license restrictions apply to commercial distribution, not to the libraries themselves.

CONCERN 01

State Machine Engine

These libraries provide the runtime engine that manages workflow states, evaluates transition guards, and fires entry/exit actions. This is the most directly relevant concern for implementing the workflow graph described in Part II.

[stateless \(SirRufo/stateless\)](#)

DELPHI NATIVE

MIT LICENSE

PRIMARY RECOMMENDATION

A direct Delphi port of the widely used .NET *Stateless* library by Nick Blumhardt. Implements a fluent, code-configured state machine. States and triggers are generically typed — enumerations, integers, strings, or any Delphi type can serve as state and trigger identifiers.

- **Generic state and trigger types** — no fixed state representation; use enums, strings, or custom types
- **Guard conditions** — anonymous method predicates on transitions; transition fires only if guard returns true
- **OnEntry / OnExit actions** — anonymous methods executed on state transitions
- **Sub-state / hierarchical support** — states can be nested; sub-states inherit parent transitions
- **Reentrant transitions** — a state may transition back to itself, triggering entry/exit actions
- **Fluent configuration API** — entire machine declared in readable code; easy to audit
- **Trigger parameters** — typed data can be attached to a trigger and passed to entry actions

Architectural fit: Guard conditions on transitions map directly to the context-requirement model. Entry actions map to "node activates → resolve context". The fluent API allows a workflow graph to be declared as readable, auditable Delphi code. Hierarchical states support nested workflows within a module.

TStateMachine (malcolmgroves)

DELPHI NATIVE

MIT LICENSE

ALTERNATIVE

An independent Delphi generic state machine framework. Acknowledges its debt to Stateless but is a separate implementation. Supports generic state and trigger types, anonymous method guards and effects, and a fluid configuration interface. Well-regarded in the Delphi community and included in multiple "awesome-delphi" curated lists.

- **Generic types** for states and triggers
- **Guard conditions** via anonymous methods
- **OnEntry / OnExit effects** via anonymous methods
- **Fluid interface** for state machine configuration
- Somewhat less feature-complete than stateless; does not document hierarchical state support

Architectural fit: Solid alternative if stateless shows compatibility issues. Simpler API may be preferable for straightforward workflows without deep nesting.

Recommendation: Start with **SirRufo/stateless**. Its direct lineage from the .NET Stateless library means its design is battle-tested, its hierarchical state support is explicitly documented, and trigger parameters cleanly support passing execution context data through transitions. Evaluate TStateMachine as a fallback if Delphi 12.2 CE compatibility issues arise.

CONCERN 02

Graph Data Structure and Workflow Storage

The workflow graph must be storable as data — independent of the state machine engine instance — so it can be designed, persisted, versioned, and loaded at runtime. There is no Delphi-native graph database library worth recommending for this purpose; the practical options are JSON or XML serialization of a custom graph model, or an embedded relational store.

Custom Graph Model + JSON Persistence

RECOMMENDED APPROACH

NO EXTERNAL DEPENDENCY

Define the workflow graph as a set of Delphi record or class types — nodes, transitions, guard descriptors, context contracts — and serialize them to JSON using one of the serialization libraries described in Concern 03. This approach keeps the graph definition human-readable, version-controllable, and loadable at application startup without an embedded database.

Workflow Graph — Minimal JSON Schema

```
{
  "workflow": "DocumentVisit",
  "nodes": [
    {
      "id": "SelectPatient",
      "entry_requires": [],
      "exit_contributes": ["PatientRecord.dfn", "PatientRecord.name"],
      "transitions": [
        { "target": "SelectTemplate", "trigger": "cmd.selectPatient",
          "requires": ["PatientRecord.dfn"], "type": "mandatory" },
        { "target": "Cancel", "trigger": "cmd.cancel",
          "requires": [], "type": "global" }
      ]
    }
  ]
}
```

```

    }
  ]
}
```

- Human-readable; reviewable by clinical informaticists without developer involvement
- Version-controllable in any source control system
- Loadable at runtime — workflow changes do not require recompilation
- Drives the state machine engine via a loader that translates graph JSON into stateless configuration calls

[GenericTree \(davidberneda\)](#)

DELPHI NATIVE

MIT LICENSE

SUPPORTING STRUCTURE

A generic tree structure implementation for Delphi. Useful for representing hierarchical workflow states (parent workflows containing nested sub-workflows) and for the module tree described in Part I. Not a graph library per se — it models trees, not arbitrary directed graphs — but covers the hierarchical structure of most healthcare workflow definitions.

- Generic typed nodes — the tree structure is independent of payload type
- Suitable for hierarchical state decomposition and module trees
- Lightweight, no external dependencies

Use for: Representing the workflow hierarchy in memory when loading from JSON. Not a replacement for JSON-based persistence.

CONCERN 03

JSON Serialization

JSON serialization serves two roles in this architecture: serializing the workflow graph definition (graph storage) and serializing the execution context and RPC request/response envelopes (HTTP communication with the RPC gateway). Delphi's built-in **System.JSON** and **REST.Json** are

functional but limited for complex object graphs. The following open source libraries are superior for this architecture's needs.

Neon – JSON Serialization Library

DELPHI NATIVE

APACHE 2.0

PRIMARY RECOMMENDATION

A mature, actively maintained Delphi JSON serialization library by Paolo Rossi. Works on existing PODO (Plain Old Delphi Object) classes without requiring changes to inheritance hierarchy. Attribute-based field mapping decouples JSON field names from Delphi property names — essential for an API layer where JSON schema must be stable regardless of internal refactoring.

- **Attribute-based JSON field naming** — JSON name is independent of Delphi property name
- **Opt-in field serialization** — only explicitly annotated fields are serialized; prevents accidental exposure of internal state
- **Supports TDateTime, generics collections** (TList, TDictionary, etc.)
- **Highly configurable** — null handling, enum serialization, naming conventions all configurable
- **No constraints on class hierarchy** — works with any existing class structure
- Used by WiRL REST framework; widely adopted in production Delphi applications
- Supports Delphi 12.x; actively maintained as of 2024

Use for: Serializing execution context objects to JSON for RPC gateway requests. Serializing and deserializing workflow graph definitions. Deserializing RPC gateway responses into typed result objects.

JsonDataObjects (ahausladen)

DELPHI NATIVE

MIT LICENSE

HIGH PERFORMANCE ALTERNATIVE

A high-performance, low-allocation JSON parser and object model for Delphi. Does not provide RTTI-based automatic serialization (you build the JSON object model manually) but is significantly faster than System.JSON for large payloads. Well-suited for parsing large Vista RPC response envelopes where performance matters.

- Substantially faster than System.JSON for parsing large JSON payloads

- Lower memory allocation; important for high-frequency RPC calls
- DOM-style API — navigate and construct JSON trees manually
- No automatic object serialization; requires hand-written mapping code

Use for: The RPC gateway HTTP request/response layer where raw parsing performance matters. Use Neon for object serialization; use JsonDataObjects for parsing large or frequent raw JSON payloads.

CONCERN 04

Supporting Infrastructure

The layered architecture requires two supporting infrastructure concerns: dependency injection (so modules can declare their dependencies rather than constructing them) and an event/message bus (so modules communicate through shared context and events rather than direct calls).

Spring4D

DELPHI NATIVE

APACHE 2.0

PRIMARY RECOMMENDATION

The most comprehensive open source library for Delphi application infrastructure. Spring4D provides a dependency injection (IoC) container, interface-based generic collections superior to System.Generics.Collections, nullable type support, and RTTI extensions. It is the de facto standard for structured Delphi application architecture and is actively maintained (version 2.0, 2023).

- **Dependency injection container** — register interfaces and their implementations; resolve dependencies automatically
- **Interface-based collections** — IList<T>, IReadOnlyList<T>, IDictionary<K,V>; enforce read/write separation across module boundaries
- **Nullable<T>** — value types with explicit null semantics; essential for execution context properties that may be unresolved
- **RTTI extensions** — richer reflection capabilities for attribute-driven architecture
- Widely adopted; extensive community documentation and examples
- Supports Delphi 12.x; current release 2.0 (November 2023)

Architectural fit: The IoC container manages module registration and lifecycle. `ICollection<T>` enforces that modules receiving context collections cannot mutate them. `Nullable<T>` precisely models the "context property present but unresolved" state identified in Part I as a critical design concern.

Delphi Event Bus (DEB)

DELPHI NATIVE

APACHE 2.0

EVENT INFRASTRUCTURE

An annotation-driven publish-subscribe event bus for Delphi. Subscribers declare interest via attributes; publishers post events without knowing subscribers. This is the correct mechanism for the inter-module communication model described in Part I: module outputs go to the event bus, other modules subscribe to events they need rather than calling each other directly.

- **Attribute-based subscriptions** — annotate methods with `[Subscribe]`; no manual registration/unregistration boilerplate
- **Background thread delivery** — events can be delivered on a background thread with `[Subscribe(TThreadMode.Background)]`
- **Decouples modules entirely** — publisher and subscriber have no compile-time dependency on each other
- Lightweight; minimal overhead for intra-process messaging

Architectural fit: Workflow state transitions publish context-change events. Module UI components subscribe to relevant context events to refresh their display. No module calls another module's methods directly — all communication is through the bus or through the shared execution context.

Delphi Mocks (VSoftTechnologies)

DELPHI NATIVE

APACHE 2.0

TESTING INFRASTRUCTURE

A mock object framework for Delphi unit testing. Given the architecture's emphasis on testing workflow logic without UI, a mocking framework is essential for injecting simulated execution contexts and RPC gateway responses into the state machine and command layers during automated testing.

- Interface and object mocking with fluent setup API

- Verify expected call sequences and parameters
- Mock the RPC gateway interface to test command/context logic without network calls
- Mock context objects to test guard conditions and transition logic in isolation

CONCERN 05

Delphi 12.2 CE Constraints

Delphi 12.2 Community Edition is free for personal, academic, and open source use. For commercial healthcare application development, a Professional or Enterprise license is required. The open source libraries listed here are independent of the Delphi edition — they compile and run identically under CE and commercial editions. Specific notes:

→ **Spring4D** requires Delphi XE or higher; confirmed working with Delphi 12.x. Install via the included Build.exe or Delphinus package manager.

→ **stateless (SirRufo)** is a pure Pascal library with no component registration; add the source path to the project library path.

→ **Neon** supports Delphi 10.3 Rio and higher; confirmed working with Delphi 12.x.

→ **JsonDataObjects** supports Delphi 2009 and higher; no version concerns.

→ **Delphi Event Bus** supports Delphi XE4 and higher.

→ **CE compiler limitation** — CE does not include full RTTI compilation for all targets; ensure RTTI settings in project options are set to include visibility modifiers (public, published) required by Neon and Spring4D reflection features.

Commercial use. If this system is deployed in a production healthcare environment, Delphi CE's license prohibits commercial distribution. All open source libraries listed use MIT or Apache 2.0 licenses, which are compatible with commercial use, but the Delphi compiler itself requires an appropriate commercial license for commercial deployment.

SUMMARY

Recommended Open Source Stack

CONCERN	LIBRARY	ROLE	LICENSE
State Machine Engine	stateless (SirRufo)	Executes workflow graph; manages state transitions, guard conditions, entry/exit actions	MIT
Graph Persistence	Custom JSON model + Neon	Stores workflow definition as JSON; loaded at startup to configure state machine	APACHE 2.0
JSON Serialization	Neon	Serializes execution context to RPC gateway JSON requests; deserializes responses	APACHE 2.0
High-perf JSON Parsing	JsonDataObjects	Parses large or frequent raw JSON payloads from the RPC gateway	MIT
Dependency Injection	Spring4D	Module registration and lifecycle; IReadOnlyList for context boundary enforcement; Nullable<T> for unresolved context properties	APACHE 2.0
Event / Message Bus	Delphi Event Bus (DEB)	Decoupled inter-module communication; context- change event delivery	APACHE 2.0
Testing / Mocking	Delphi Mocks	UI-free testing of workflow logic, command execution, and context handling	APACHE 2.0

CONCERN	LIBRARY	ROLE	LICENSE
Hierarchical Structure	GenericTree	In-memory representation of hierarchical workflow and module trees	MIT

Key observation. All eight concerns can be addressed with open source libraries that have zero external runtime dependencies and compile cleanly as Object Pascal source included in the project. The result is a self-contained application with no external service dependencies beyond the VistA RPC gateway — appropriate for a clinical environment with strict network and deployment controls.